JobMatch AI

AI-Powered Job Recommendation System

Big Data Analysis Project • MongoDB · Kafka · Spark · ML

Milena & Sona

Project Overview

Goal

Build a real-time job recommendation web app that learns from user behavior

End-to-End Pipeline

Web → Kafka → Spark Streaming → ML → MongoDB → Web

AI Engine

TF-IDF + Cosine Similarity scores user skills & click history to rank jobs

System Architecture

User clicks a job → event sent to Kafka → Spark Streaming processes it → ML model generates recommendations → stored in MongoDB → displayed on web page

Technology Stack

🎨 Frontend

- HTML5 / CSS3 / JavaScript
- Jinja2 Templates
- Sora Font, Modern UI Design

⚙️ Backend

- Python Flask
- Session-based Auth
- REST API endpoints

📡 Streaming

- Apache Kafka (job-clicks topic)
- Kafka Producer in Flask
- Real-time event pipeline

⚡ Big Data

- Apache Spark Structured Streaming
- foreachBatch processing
- Distributed computation

🗄️ Database

- MongoDB Atlas (Cloud)
- 3 Collections: users, jobs, recommendations
- NoSQL document store

🤖 ML / AI

- TF-IDF Vectorization
- Cosine Similarity
- 60% Skills + 40% Click scoring

ML Algorithm – How Recommendations Work

1

Collect Input

User skills (from registration) + last clicked job title

2

TF-IDF Vectorization

All job skill texts + user input → TF-IDF matrix

3

Cosine Similarity

Compute similarity between user vector and each job

4

Weighted Scoring

Final Score = Skills Match × 0.6 + Click Match × 0.4

5

Top 3 Results

Sort by score, save top 3 recommendations to MongoDB

⭐ Score Formula

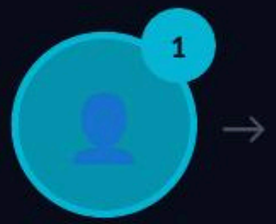
```
score = (skills_sim × 0.6)
      + (click_sim × 0.4)
```

Skills Match 60%

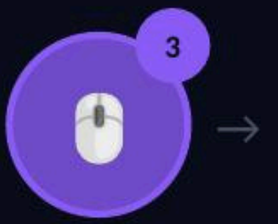
Click Similarity 40%

Result: Top 3 jobs ranked by relevance score

Real-Time Data Flow

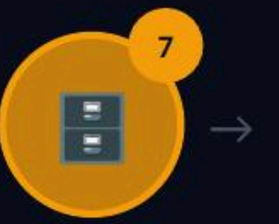


User registers with skills



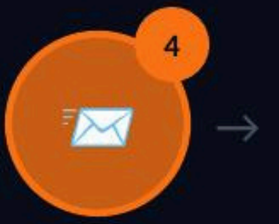
Clicks 'I'm Interested'

Spark reads Kafka stream

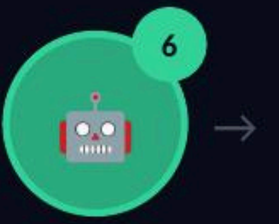


Saves top 3 to MongoDB

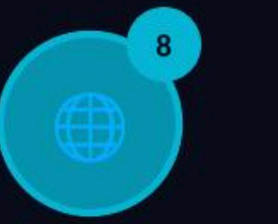
User browses job listings



Flask sends event to Kafka



ML calculates scores



Page shows recommendations

Recommendations page auto-refreshes every 5 seconds to show latest AI results

MongoDB – Data Model

users

username	String	Unique identifier
password	String	Plain text
skills	Array	e.g. ['Python', 'SQL']

jobs

job_id	Integer	Unique job ID
title	String	e.g. ML Engineer
company	String	e.g. YerevaNN
skills	String	Space-separated keywords

recommendations

username	String	User reference
recommended_jobs	Array	Top 3 job objects
based_on	String	Last clicked title
user_skills	String	Snapshot of skills

Web Application – Key Screens

Register / Login

- Username + password auth
- User enters their skills
- Skills stored in MongoDB
- Used as ML input later

Job Listings

- All available jobs displayed
- Each card shows required skills
- Click 'I'm Interested' to train AI
- Click event sent to Kafka

AI Recommendations

- Top 3 jobs ranked by AI
- Shows % match score
- Based on: skills + click history
- Auto-refreshes every 5 seconds